AdsKG-Agent: an AI Assistant That Finds the Right Ad Data

Capstone design doc · AI Engineering Cohort · Anshika Chaurasia (solo) · October 2026

In one sentence: an AI assistant that answers ad-performance questions by first *finding* the right table (search), then *connecting* it to related tables (knowledge graph), then writing and running the database query — and remembering what worked. This capstone builds the **"What happened?"** stage; "Why", "What next" and "Act" are planned future versions (Section 8).

1. Problem Statement

Ad teams ask questions like *"What was the click-through rate (CTR) for brand 95471 yesterday?"* or *"How many unique people saw campaign 83237 in the last 6 hours?"* The data sits in many tables that look almost the same: hourly vs. daily vs. weekly totals, all traffic vs. suspicious clicks removed, total views vs. unique people, live-but-incomplete vs. final. Pick the wrong one and the number *looks* right but is wrong, and nobody notices.

Goal: given a question in plain English (and the earlier chat), pick the correct table(s) and how to join them, write the database query (SQL), run it, and answer with warnings (e.g. "today's data is incomplete"). **Not in this version:** explaining *why* numbers changed, recommendations, actions, or changing any data.

My hypothesis: the hard part is not that the AI can't *see* all the tables, but that it can't *tell look-alike tables apart* or misses the tables needed to connect them. Search plus a table map should fix that.

Targets (set before building): best version ≥ 75% correct on the test questions; ≥ 10 points better than basic search on the medium and tricky questions; ≥ 80% correct on follow-up questions with memory; typical answer ≤ 10 s; ≤ $0.02 per question.

Why it matters: my company's ads assistant answers "what" questions today; this project tries out the search, knowledge-graph, memory and MCP techniques it doesn't have yet, built from scratch on public data.

2. Data

- **Source:** Taobao (Alibaba) ad dataset, public on Kaggle: 26.6M ad views and 1.37M clicks (CTR 5.14%) from 1.14M users over 8 days in May 2017; ad details for 847k ads (423k campaigns, 256k advertisers, 99.8k brands, 6.8k categories, price) and 1.06M user profiles (gender, age group, spending level, city size).
- **Prep (DuckDB):** fix problems found on load (29% of ads have no brand; 1.5M views have no user profile), then build ~28 tables in standard warehouse style, including the look-alike families above. Every rule (e.g. what counts as a suspicious click) is fixed and written down *before* any test question is written. "Value" = clicks × item price, clearly labelled an estimate (the data has no spend or revenue). IDs stay as numbers; no made-up names.
- **Fixed "now":** the data is from 2017, so "now" = the last hour of the data (13 May 2017, 23:59 China time). "Live" tables hold the last 6 hours; "final" daily tables stop at yesterday, like a real 1-day delay.
- **Table cards:** each table gets a short description (what it holds, time grain, freshness, 3 example values, how it joins), drafted by AI from the table itself, then checked and frozen by me. Cards go into Qdrant; the tables, columns, metrics, business terms and their connections go into Neo4j.
- **Extra tables for the bigger-catalog test:** public TPC-DS and BIRD tables are added to the catalog only (never part of a correct answer) to grow it from ~28 to ~100 and ~250 tables.

3. How It Works (Architecture)

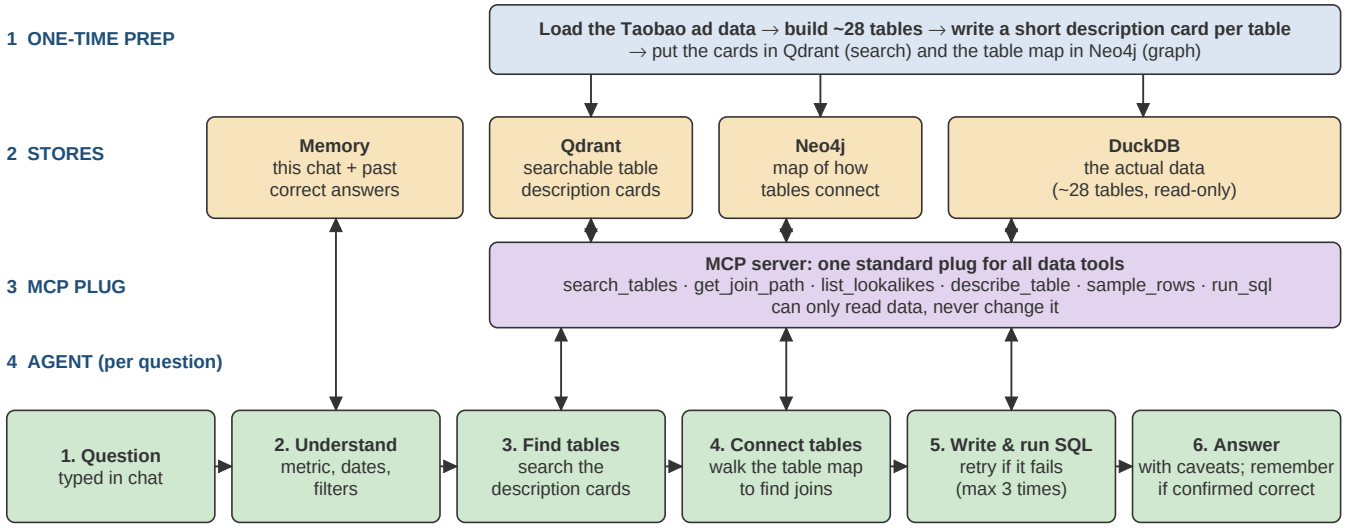

Figure 1. Prep happens once. For each question the agent understands it, finds and connects tables through the MCP plug, runs the query and answers. Memory handles follow-ups like "now split that by age". The test harness sends every test question through this same flow.

Part	What it does
Table map (Neo4j)	Things: tables, columns, metrics (CTR, reach), business words ("views" = impressions), advertiser → campaign → ad hierarchy, past correct queries. Links: "joins with", "look-alike of (differs by grain / freshness / traffic filter)", "hourly rolls up to daily", "metric is computed from column"
Map-walking rules	Find the shortest route between the tables search found (≤ 3 steps) and add their look-alikes so the AI can choose; never more than 8 tables, to avoid flooding it
MCP plug	One small server (FastMCP) offering search_tables, get_join_path, list_lookalikes, describe_table, sample_rows, run_sql. Every version uses it, and Claude Desktop or Cursor can use it unchanged for the demo
Safety	Inside the MCP server, so nothing can bypass it: read-only database, one SELECT query only (checked with sqlglot), max 1,000 rows, 20-second timeout, every call logged
Agent harness	Rules around the agent: ≤ 8 steps, ≤ 3 retries, cost limit per question, only the top 8 table cards (~2k tokens) in its context, long chats summarised after 6 turns, says "not sure" instead of guessing
Memory	Short-term: remembers this chat. Long-term: saves questions it answered correctly and reuses them as examples, plus user preferences. During testing it learns only from practice questions

Part	What it does
Test harness	One script runs any version on any question set (settings in a config file), saves every step (Langfuse), and produces all scores and charts, so every result can be re-run with one command

4. What I Compare (start simple, add one idea at a time)

Version	What it does	What it shows
A. No search	Give the AI every table description at once, then write SQL	The simple baseline
B. Basic search	Search the cards by meaning (Qdrant) and give the AI only the best few	Does search help?
C. Smart search	B + keyword-and-meaning search, re-ranking, and rewriting the question first (each piece switched on and off to see what helps)	Do advanced RAG tricks help?
D. Search + map	C, then follow the Neo4j map to add connecting tables and look-alikes	Does the knowledge graph help?
E. Agent + memory	An agent that uses the MCP tools step by step and retries; run with and without memory	Do the agent and memory help?
E′. Stretch	Same agent with ready-made generic MCP servers for Neo4j, Qdrant and DuckDB instead of my tools	Custom vs. generic tools

The final choice is the version with the most correct test answers; if two are statistically tied, the cheaper and faster one wins.

5. How I Measure Success (Evaluation)

- **Test questions:** 120 questions with known correct answers and the correct SQL (run and checked by hand), 40 at each level: easy (one table), medium (tables must be joined), tricky (look-alike tables). Written *before* versions B–E exist, so I can't tune them to pass; if possible a classmate writes 20 without seeing my system. 40 for practice, 80 for the final test, plus 20 short follow-up conversations (5 practice / 15 test). All published.
- **Scores:** right tables picked (precision/recall, F1); right final answer (same result rows, ignoring order and tiny rounding); right joins; how often a retry was needed; time, tokens and cost per question; follow-up accuracy with memory on vs. off.
- **Bigger-catalog test:** scores for A–D as the catalog grows from 28 to ~100 to ~250 tables, showing when "give it everything" stops working.
- **Not just luck:** every version runs 3 times; I report ranges (95% bootstrap confidence intervals) and check each step-up A→B→C→D→E with a paired significance test (McNemar).
- **AI grader (LLM-as-judge):** a different, larger AI that doesn't know which version wrote the answer scores relevance, correct warnings, and faithfulness to the query result (1–5). I grade 30 answers myself and report how well it agrees with me (Cohen's kappa).
- **Failure review:** at least 30 failures sorted into types (wrong look-alike table, missing connecting table, wrong join, too many tables, made-up column, wrong time window) for the README.

6. Tools and Why (Framework Justification)

Tool	Job in this project	Why this one (alternative considered)
DuckDB	Holds the ad data	Fast on a laptop with 26M rows, no server (Postgres)
Qdrant	Vector database for the table cards	Keyword + meaning search and filters built in (Chroma, pgvector)
Neo4j	Graph database for the table map	Easy "route from A to B" queries; industry standard. At 28 tables a Python library (NetworkX) would do; Neo4j is chosen for the 100–250-table test and for use at work
LangGraph	Runs the agent, its rules and memory	Retry loops and memory built in (Pydantic AI, smolagents)
MCP (FastMCP)	Standard plug for the data tools	Same tools for every version and any AI app; safety in one place
Langfuse	Records every step of every run	Open source; makes failures easy to inspect (LangSmith)
OpenAI models	Write SQL; a different model grades	Course-provided key; embeddings text-embedding-3-small; open-source bge reranker

7. Risks and What I'll Do

- **Version A wins at 28 tables** → that's a real finding; the bigger-catalog test shows where search becomes necessary.
- **I wrote the questions myself** → questions and rules frozen first, classmate-written subset, results shown per level, everything published. **AI-written table cards give hints** → checked, frozen, identical for all versions.
- **Many new tools at once** → build in order A → B → D → E first (each one works before the next), then C's on/off tests, E′ and the 250-table test. **MCP adds delay** → measured by the test harness.
- **Simulated "now" and estimated value** → clearly stated; no question claims real spend or revenue.

8. Future Versions: from "What" to "Act"

Stage	Question it answers	What gets added	Reuses from this capstone
1. What (this capstone)	"What was CTR yesterday?"	Finding tables, SQL, memory	—
2. Why	"Why did CTR drop 18%?"	Break the change down by campaign, placement, audience; metric tree (revenue = views × CTR × conversion) added to the map	Map, MCP tools, test harness
3. What next	"What should I check or change?"	Playbooks of known causes and fixes searched with RAG; ranked suggestions with evidence	Search, memory
4. Act	"Move budget from A to B"	Optimisation to propose budget moves; a person must approve before anything runs; full audit log	MCP plug (new write tools), safety, harness